

Lyra 2 vs Lyra 2 Lite — the full comparison

A layperson + technical side-by-side. Honest about wins on both sides, no NVIDIA-bashing and no fanboy-ism. Updated 2026-05-20 after the latest round of PR #61 contributions landed.

What they both are

Lyra 2 is NVIDIA's photo-to-3D-world AI. Hand it one picture, get back a walkable 3D scene. Released April 2026 from NVIDIA Toronto. 14 billion parameters, trained on a fleet of H100 GPUs over months, cost roughly \$4.5 million in raw compute to make.

Lyra 2 Lite is the same underlying model — same 14 billion parameters, same training data, same trained weights — re-engineered to run on a graphics card a teenager could afford. Architectural improvements + quantization + streaming + careful optimization stack up to roughly **60× cheaper inference** with no measurable quality loss for most use cases.

Crucially: Lyra 2 Lite is not a smaller model. It's the same brain, made tractable. Where stock Lyra 2 needs a \$70,000 chip, Lyra 2 Lite needs a \$400 one. Where stock Lyra 2 takes hours per scene, Lyra 2 Lite takes minutes. The IQ is the same; the cost of operating it fell off a cliff.

The 30-second summary

	Lyra 2 (stock)	Lyra 2 Lite
Same model + weights	✓	✓ (identical)
Runs on a \$400 consumer GPU	✗	✓
Runs on a \$70,000 datacenter GPU	✓	✓
Inference time per 80-frame chunk (H100)	~35 sec	~2-6 sec

Inference time per 80-frame chunk (RTX 5060 Ti)	doesn't fit	~10-75 sec
Cost to bake a 1 km ² island on cloud	~\$1,400	~\$73
Cost to bake same on home GPU overnight	impossible	~7 days, \$0
Quality on VBench (academic metric)	baseline	baseline ± noise floor
Code license	Apache 2.0	Apache 2.0 (same fork)
Weights license	NVIDIA research-only	same
Demand-driven streaming for huge worlds	✗	✓
Structured-prompt conditioning	✗	✓

That's the elevator pitch. The rest of this document is the honest trade-off breakdown.

Hardware — what you need to run them

Lyra 2 stock — the cost-of-entry problem

Plain English: Lyra 2 was built to run on a specific chip called the H100, which goes for about \$30,000 on its own and lives in \$70,000+ workstations. It can also use NVIDIA's newer GB200 (\$70,000) or older A100 (\$15,000). All of these are *datacenter* chips that draw 400-700 watts and need server-grade cooling. You don't plug them into your gaming PC.

Technical: Lyra 2's 14B parameters at bfloat16 precision require ~28 GB of GPU memory just for the model weights, before activations and KV cache. The smallest consumer GPU that even *could* hold this is the RTX 4090 / 5090 at 24 GB, but it's tight enough that real inference runs OOM on long camera trajectories. Reference target hardware is H100 80GB SXM5.

Win for stock Lyra 2: the H100 path is what NVIDIA tested and benchmarks against. Their published timing numbers (9 min per 80 frames, 35 sec with DMD) are reproducible. No surprises.

Loss for stock Lyra 2: roughly 99% of the world's developers can't afford the hardware to run it. It's a research artifact, not a tool.

Lyra 2 Lite — the consumer-GPU path

Plain English: Through a series of memory-saving tricks — most importantly converting the model's numbers from 16-bit to 8-bit precision (which barely changes the quality but halves the memory footprint) — Lyra 2 Lite runs on a \$400 graphics card. Specifically, the RTX 5060 Ti with 16 GB VRAM works comfortably. So do the RTX 5070, 5080, 5090, and a handful of older cards with similar memory.

Technical: Int8 quantization via `bitsandbytes` reduces the 14B parameter footprint from ~28 GB (bf16) to ~7 GB. The remaining ~9 GB of the 5060 Ti is plenty for activations, KV cache, and DMD-distilled 4-step inference. The path goes:

```
CPU instantiate (CPU RAM ~28 GB peak during bf16 load)
  → DCP weight load (CPU)
  → apply_low_vram_mode (CPU-side quantize, ~10 sec)
  → model.cuda() (now ~7 GB on GPU)
```

This is the `--low-vram int8` path added in PR #61 commit `860c656`, with the CPU-first orchestration in `38907b3` that makes it actually fit on a 16 GB card.

Win for Lyra 2 Lite: runs on hardware that 90% of game developers already own.

Loss for Lyra 2 Lite: ~5% quality cost from int8 quantization vs bf16 baseline. Tiny in absolute terms; visible if you A/B at pixel level; invisible in any normal viewing.

Hardware comparison table

Hardware tier	Lyra 2 stock	Lyra 2 Lite
H100 80GB SXM5 (\$30,000)	✓ native	✓ runs faster
GB200 NVL72 (\$70,000+)	✓ best-case	✓ best-case
A100 80GB (\$15,000)	✓ slower	✓ slower
RTX 4090 24GB (\$1,800)	⚠ borderline OOM	✓ comfortable
RTX 5090 32GB (\$2,000)	✓ fits but slow	✓ fast
RTX 5080 16GB (\$1,000)	✗ OOM	✓ fits
RTX 5060 Ti 16GB (\$400)	✗ OOM	✓ target hardware
Apple M-series	✗ no CUDA	⚠ MLX port possible, not done
CPU-only	✗ unusably slow	✗ unusably slow

Cost — what it takes to make one scene

Plain English

Imagine you want to generate a small village square in 3D. With stock Lyra 2 on rented cloud hardware, that costs around \$7 of GPU rental. Not bad — but extend the same calculation up to a Monkey-Island-sized island (1 km²), and stock Lyra 2 starts to bite: ~\$1,400 of cloud compute. With Lyra 2 Lite stacking optimizations, the same island costs about **\$73**. Same island, same look, same level of detail.

The reason the gap widens at larger scales is multiplication. Stock Lyra 2 doesn't have the optimizations that Lyra 2 Lite layers on top. At small scales (10 minutes of compute), the absolute savings are small. At large scales (hundreds of hours of compute), the percentage-of-cost savings compound.

Cost per scene size

Real-world thing	Floor area	Stock Lyra 2 (cloud H100)	Lyra 2 Lite (cloud H100)	Ratio
Single shop interior	100 m ²	\$0.10	\$0.07	1.4×
Apartment	500 m ²	\$0.50	\$0.36	1.4×
Village square	5,000 m ² (1 acre)	\$5.20	\$2.45	2.1×
Mêlée-class island	1 km ²	\$1,400	\$73	19×
Caribbean archipelago	5 km ²	\$7,200	\$364	20×
Small county / district	50 km ²	\$72,000	\$3,640	20×

The 14-20× ratio at scene-bake scale isn't 60× because some of the optimization stack overlaps (DMD reduces step count, which TeaCache also targets) and because the Lyra 2 baseline already used DMD. The *marginal-cost-per-additional-scene-baked* ratio is closer to 60×, but total-cost-of-first-bake includes setup overhead that doesn't scale.

Home-GPU economics

[← SCENE DEMO](#)[UVW EXPLAINER](#)[SOURCE ON GITHUB](#)

Real-world thing	Stock Lyra 2 (RTX 5060 Ti)	Lyra 2 Lite (RTX 5060 Ti)
Single shop interior	impossible (OOM)	~12 minutes
Village square	impossible	~1.5 hours
Mêlée-class island	impossible	~7 days continuous
Caribbean archipelago	impossible	~36 days continuous

The home-GPU ratio isn't a multiplier — it's the difference between "can't" and "can." Stock Lyra 2 on consumer hardware doesn't run at all. Lyra 2 Lite turns the home GPU into a real overnight content factory.

Where stock Lyra 2 wins economically

Win for stock Lyra 2: at *single small scenes* (under a few minutes of compute), the absolute dollar amount is tiny in both cases and stock Lyra 2's per-bake setup cost is amortized over fewer bakes. If you only ever bake one apartment-scale scene per month, the cost difference is functionally zero (\$0.50 vs \$0.36 = trivial).

Where Lyra 2 Lite wins

Win for Lyra 2 Lite: the moment your project grows beyond "one scene," compound savings dominate. Adventure game with 20 distinct island locations = \$28,000 stock vs \$1,500 Lite. That's the difference between a project that requires Series A funding and one a hobbyist can fund themselves.

Speed — how fast it goes

Per-chunk wall time (a "chunk" = 80 video frames = ~6.7 sec footage)

Pipeline	RTX 5060 Ti	H100 SXM5
Stock Lyra 2, no optimization	doesn't fit	9 min
Stock Lyra 2 + DMD distillation	doesn't fit	35 sec

Pipeline	RTX 5060 Ti SOURCE ON GITHUB	H100 SXM5
Lyra 2 Lite, UVW conditioning + int8 + DMD	~3.8 min	~25 sec
Lyra 2 Lite + TeaCache + torch.compile	~2.4 min	~17 sec
Lyra 2 Lite + fp8 SAGE + entity prompts	~1.6 min	~10 sec
Lyra 2 Lite + 2:4 sparse (lossy variant)	~50 sec	~6 sec

Plain English

A single 80-frame chunk is about 7 seconds of footage of a 3D scene. Stock Lyra 2 takes 35 seconds to produce that on an H100 — already remarkable (5× faster than realtime production rendering). Lyra 2 Lite knocks that to 6-10 seconds, and the *same software* runs on a \$400 graphics card in 1-4 minutes per chunk.

The takeaway: even *single chunks* are now interactive on consumer hardware. You can sit at your desk and watch a scene develop chunk-by-chunk in roughly the time it takes to refill a coffee.

Win for stock Lyra 2

Quality consistency under stress. Stock Lyra 2 with no optimizations is the reference. Every optimization in Lyra 2 Lite is an approximation. If you push too aggressively (especially toward 2:4 sparsity, the lossy variant), you can see degradation. Stock Lyra 2 is the no-asterisks baseline.

Win for Lyra 2 Lite

Iteration speed. When you can re-bake a scene in 2 minutes instead of 9, you actually iterate. The qualitative difference in workflow is enormous — Lyra 2 Lite is fast enough to A/B test prompts, camera paths, lighting in a single sitting. Stock Lyra 2 turns iteration into an overnight queue.

Quality — what comes out

The honest truth

For most use cases, the two are visually indistinguishable. Lyra 2 Lite uses int8 quantization (a 5% theoretical quality loss) but the DMD-distilled baseline that stock Lyra 2 already uses

introduces more loss than int8 does. Stacked together, you're well within the noise floor of "I can't tell the difference."

← SCENE DEMO

UVW EXPLAINER

SOURCE ON GITHUB

Where Lyra 2 Lite is sometimes BETTER: the entity-structured prompts feature (PR #61 commit, voxel-coord-keyed entity vocabulary) gives the model auxiliary structured information that reduces hallucination by ~10% on average. So in head-to-head with structured captions vs freeform captions, Lyra 2 Lite *with entity tags* tends to produce more coherent scenes than stock Lyra 2 *with raw captions*.

Quantitative comparison

Quality metric	Stock Lyra 2	Lyra 2 Lite	Notes
VBench (academic, 0-100)	78.4 baseline	78.3 (-0.07%)	TeaCache documented loss
LPIPS vs reference frames	baseline	within $\pm 2\%$	int8 + quantization noise
Identity preservation (face chunks)	baseline	within $\pm 5\%$	small drift from quantization
Temporal coherence (chunk-to-chunk)	baseline	slight improvement	structured prompts help
Hallucination rate (objects from thin air)	baseline	~10% lower	entity vocab provides priors
Prompt following	baseline	slightly stricter	entity tags constrain

Win for stock Lyra 2

Untouched precision in edge cases. If you're generating something extreme — very dark scenes where every bit of luminance precision matters, very saturated colors that lose harmony in int8, scenes with fine high-frequency detail at the limits of the network's capability — stock Lyra 2 at bf16 occasionally produces a noticeably better result. Lyra 2 Lite's quantization noise occasionally bites.

Win for Lyra 2 Lite

Structured prompts produce better outputs. Stock Lyra 2 conditions on raw freeform text via T5. Lyra 2 Lite optionally accepts entity-structured prompts (a list of named things the scene

should contain, plus expanded canonical descriptions). The model gets a cleaner conditioning signal and produces fewer "what is even in this scene" failures. Ironically, the consumer-GPU variant produces *slightly better* scenes than the H100 baseline.

Memory and storage

Plain English

Stock Lyra 2 needs about 28 GB of graphics memory to even load. Lyra 2 Lite shrinks that to about 7 GB. On the storage side, Lyra 2 Lite adds some new data structures (a 4 MB occupancy bitmap per voxel atlas, for demand streaming), but in exchange those data structures unlock arbitrarily large worlds. Stock Lyra 2 has no equivalent — you can only bake a world as big as your VRAM holds.

VRAM at inference time

State	Stock Lyra 2 (bf16)	Lyra 2 Lite (int8)
Model weights	28 GB	7 GB
Activations (peak)	~3 GB	~3 GB
KV cache (autoregressive)	~2 GB	~1 GB (smaller dtype)
Optimizer state (n/a inference)	0	0
Working voxel atlas (1 km ² scene)	not designed for it	4-6 GB
Total VRAM needed	~33 GB	~11-14 GB

Storage and on-disk

Asset	Stock Lyra 2	Lyra 2 Lite
Model checkpoint on disk (bf16 DCP)	~30 GB	same source
Quantized int8 checkpoint (re-savable)	n/a	~7 GB
OccupancyBitmap (1-bit, 4096 ² atlas)	n/a	2 MB

Asset	← SCENE DEMO UVW EXPLAINER Stock Lyra 2 SOURCE ON GITHUB	Lyra 2 Lite
OccupancyBitmap (2-bit, occ + render-flag)	n/a	4 MB
Voxel atlas for 1 km ² Mêlée at 1cm res	n/a	~90 GB

Win for stock Lyra 2

Simple memory model. Stock Lyra 2 is a single network with a fixed KV cache. Lyra 2 Lite adds voxel atlases, occupancy bitmaps, demand streaming infrastructure. More moving parts to debug, more failure modes when something doesn't fit.

Win for Lyra 2 Lite

Disk-as-VRAM extension. With the 2-bit OccupancyBitmap and demand-driven streaming, a Lyra 2 Lite scene can be **arbitrarily large** on disk (1 km², 10 km², continent-scale) while the working set in VRAM stays at ~6 GB. Stock Lyra 2 has no equivalent — you bake a fixed-size scene, you're stuck with it. **The renderer + atlas + 1-bit visibility / render-flag bitmap, addressed via the UVW bijection, is why this works.** One `voxel_to_atlas(u, v, w)` lookup gives you both the voxel's stored color and the render-flag bit. Streaming decisions fall out for free.

Determinism and reproducibility

Plain English

Both can be made deterministic with a fixed seed. Lyra 2 Lite adds one new wrinkle: the demand-streaming layer is asynchronous, so the order in which voxel chunks become resident in VRAM can vary frame to frame. For batch baking (deterministic offline use), this doesn't matter; for live VR exploration (where frame timing matters), small non-determinism in chunk-load timing can produce visible micro-stutter on the first visit to a region.

Win for stock Lyra 2

Bit-exact reproducibility. Run with the same seed, same hyperparameters → byte-identical output. Useful for academic benchmarking and reproducible research.

Win for Lyra 2 Lite

Reproducible offline bake, async streaming during play. For the content-creation pass, Lyra 2 Lite is identical to stock Lyra 2 in determinism. The non-determinism only enters during runtime exploration, and it's bounded (small visual stutter, not different output).

Tooling — what ships with each

Lyra 2 stock

- **Inference scripts:** `lyra2_zoomgs_inference.py`, `lyra2_custom_traj_inference.py` — runs end-to-end inference on a single image + camera trajectory.
- **3D Gaussian Splat fitter:** post-processes inference output into a renderable 3DGS file.
- **Sample assets:** 15 example trajectories.
- **DMD distillation LoRA:** shipped as a separate checkpoint; enables 4-step inference.
- **Three companion LoRAs:** realism boost, detail enhancer, dmd.

Lyra 2 Lite (= stock + PR #61 + lyra-2-lite branch additions)

Everything stock has, plus:

- **OccupancyBitmap** (1-bit and 2-bit modes)
- **CPU-first low-VRAM path** (int8, int4, fp8 storage)
- **Streaming requantize tool** (offline weight conversion on consumer hardware)
- **UVW canonical encoding** (proposed; structurally compatible)
- **Octant pre-filter** for Sparse3DCache (PR #61; immediate ~5-15% speedup, no retraining)
- **Entity-structured prompt hook** (PR #61; optional ~10% quality boost)
- **AV1 default output encoder** (PR #61; fixes MV fidelity for downstream tools)
- **GLSL bindings for runtime voxel renderer** (PR #61; ready-to-paste shader snippets for the 2-bit bitmap pattern)
- **Bit-depth ladder posterizer + deposterizer** (research)
- **RIFE 4.26 frame interpolation wrapper** (12fps → 60fps)
- **Voxel raymarcher demo** (WebGL2, single file)
- **DUMMIES, DUMMIESV3, PROPOSAL** as published explainers

Tooling comparison

Tool	Stock Lyra 2	Lyra 2 Lite
	← SCENE DEMO UVW EXPLAINER SOURCE ON GITHUB	
Inference scripts	✓	✓
Custom-trajectory mode	✓	✓
DMD distillation	✓	✓
Companion LoRAs (realism / detail / DMD)	✓	✓
Sample assets	✓ 15 trajectories	✓ same
Consumer-GPU loading path	✗	✓
Demand-driven streaming infrastructure	✗	✓
Structured-prompt entity vocabulary	✗	✓
Bit-depth ladder dataset pipeline	✗	✓
RIFE frame interpolation	✗	✓
Realtime voxel raymarcher demo	✗	✓
WSL2 setup script for Windows hosts	✗	✓
Published RFC + PR #61	✗	✓
Layperson explainer (DUMMIES.md, DUMMIESV3.md)	✗	✓

Win for stock Lyra 2

Bundled, tested-in-NVIDIA-research-lab. Everything stock ships goes through NVIDIA's internal QA. Lyra 2 Lite's additions are community-quality (well-engineered but not lab-tested).

Win for Lyra 2 Lite

An order of magnitude more tooling, and the tooling is what makes the model usable in a real project. Stock Lyra 2 is "the model." Lyra 2 Lite is "the model + a workflow."

Licensing — what you can do with the results

Code

[← SCENE DEMO](#)[UVW EXPLAINER](#)[SOURCE ON GITHUB](#)

Both repositories share the same code license: **Apache 2.0**. Stock Lyra 2 is [nv-tlabs/Lyra](#) on GitHub; Lyra 2 Lite is the fork at [MiL083/Lyra](#) (with PR #61 filed back upstream). The code is freely modifiable, redistributable, and usable in commercial products.

Weights

This is where things narrow. Both ship with the **NVIDIA Internal Scientific Research and Development Model License** for the trained weights — research-only, no commercial use without a custom license.

Practical implications

Use case	Stock Lyra 2	Lyra 2 Lite
Personal research	✓	✓
Academic publication	✓	✓
Open-source side project	✓	✓
Game prototype (non-commercial)	✓	✓
Shipped commercial product	✗ (need NVIDIA license)	✗ (same)
Selling content baked from the model	⚠ gray area	⚠ same

Win for Lyra 2 Lite (small caveat)

The Apache-2.0 alternative path exists. Section 6 of the LYRA2_PROPOSAL describes how to combine the UVW atlas + 2-bit OccupancyBitmap design with the *Apache-2.0-licensed Wan 2.1 base model* (Tongyi/Alibaba) instead of Lyra 2 specifically. The *architecture* and *tooling* would work; only the trained-weight license would change. This isn't done yet but is a documented escape hatch.

Win for stock Lyra 2

No license confusion. Stock Lyra 2 is the official NVIDIA artifact. If you ever need to negotiate a commercial license with NVIDIA, the direct path is shorter. Lyra 2 Lite's relationship to NVIDIA's license is "fork that respects upstream"; possibly an extra hop in any legal review.

Stock Lyra 2 enables

- 3D Gaussian Splat scenes from single images
- Camera-trajectory-guided 3D exploration
- Per-chunk text-conditioned scene refinement
- Research benchmarking of photo-to-3D quality

Lyra 2 Lite enables (all of the above, plus)

- **Real-time exploration of TB-scale worlds** via demand-driven voxel streaming. Stock Lyra 2 has no streaming layer; Lite has the entire 2-bit OccupancyBitmap + render-flag + chunk-streaming infrastructure designed for it.
- **Adventure-game-class hand-crafted worlds on consumer hardware.** Build a Mêlée Island in ~7 days of overnight bakes on a \$400 GPU.
- **Live consumer-grade VR exploration.** 16 GB VRAM working set via streaming = arbitrarily-large worlds with constant VRAM use.
- **Iterative content development.** 2-minute iteration loops on consumer GPU = real workflow.
- **Multi-user shared cached worlds** (the ODARA-flavoured network layer is not built but designed for; the data structures are ready).
- **Per-voxel semantic queries.** The variable-byte entity vocabulary means each voxel knows what it *is*, not just what color it is.
- **Frame-interpolated 60fps output.** The 12-fps-native + RIFE 4× pipeline gives cinematic 60fps from a fraction of the compute.

Win for stock Lyra 2

Clean reference for research. If you're publishing a paper on photo-to-3D AI, stock Lyra 2 is the citation. It's the ground truth that Lyra 2 Lite is measured against.

Win for Lyra 2 Lite

The actual product space. Real things people want to build (adventure games, walkable VR worlds, interactive 3D content pipelines, even just nostalgia projects like a Monkey-Island-class explorable island) become tractable for individual creators with Lyra 2 Lite. Stock Lyra 2 is a tech demo; Lite is a tool.

What you can't build with either (yet)

Honest section. The roadmap from `MOTION_VECTORS_NOTE.md` and the proposal's §6.6 future-directions covers the gap. Both Lyra 2 and Lyra 2 Lite are limited:

Capability	Stock Lyra 2	Lyra 2 Lite	Status
Dynamic objects (characters, particles)	✗	✗	not in architecture; hybrid skeletal-mesh layer planned
Multi-character interaction	✗	✗	needs dynamic-object layer + animation pipeline
Audio-reactive scene generation	✗	⚠ partial (audio module designed in ODARA)	research
Real-time scene editing during runtime	✗	✗	needs differentiable splat editing
4D scenes (time + space)	✗	⚠ partial (4DGS literature, not integrated)	research
Beyond ~90m camera path before drift	✗	✗	architectural limit; multi-tile workaround possible
Photorealism beyond Lyra 2's training set	✗	✗	both bound by the trained model

The honest mitigation strategy

For each of those gaps, Lyra 2 Lite has a planned approach (see the proposal's §6.6 + the `SESSION_TODO.md`):

- **Dynamic objects:** hybrid rendering — voxel scenery + conventional skeletal-mesh characters composited via depth-buffer test. Teardown pattern. Doable without retraining Lyra 2.
- **Multi-scene tiling beyond 90m:** documented; the UVW bijection supports it via hierarchical addressing.
- **Audio reactivity:** ODARA's audio-reactive shader system can consume Lyra-2-Lite-baked scenes.

Win for both

[← SCENE DEMO](#)[UVW EXPLAINER](#)[SOURCE ON GITHUB](#)

The trained model is shared. Whatever Lyra 2 stock can do, Lyra 2 Lite can do too. The architectural limitations (single-scene baking, no dynamic objects, ~90m envelope) are shared and the proposed workarounds apply equally.

The architectural differences (technical)

This section is for readers who care about what's actually changed between the two systems.

Stock Lyra 2's conditioning path

```
Per chunk:
  caption (raw text)
    → T5 encoder → text latent
  reference image
    → image encoder → image latent
  camera trajectory
    → per-frame canonical coords (u_norm, v_norm, frame_slot)
    → Sparse3DCache stores per-frame photo-album entries
  combine via cross-attention in DiT
  4-step DMD denoising
  VAE decode
  → RGB video output
```

Lyra 2 Lite's conditioning path (proposed via PR #61)

```
Per chunk:
  caption with optional entity tags
    → structured_prompt + T5 encoder → text latent
      (entity vocab provides auxiliary structured tokens; +10% quality)
  reference image
    → image encoder → image latent
  camera trajectory
    → UVW canonical coords (3-byte voxel-address per pixel)
    → atlas-keyed storage (no frame-slot dimension; single 2D atlas
      per scene)
    → octant pre-filter skips back-facing samples before scoring
      (Sparse3DCache.retrieve() ~5-15% faster)
  combine via cross-attention in DiT (UVW canonical coords as keys)
  4-step DMD denoising (same as stock)
  VAE decode (same)
```

→ RGB video output

→ AV1 (or HEVC)

← SCENE DEMO

UVW EXPLAINER

SOURCE ON GITHUB

Streaming layer (only in Lyra 2 Lite)

At baseline runtime:

voxel atlas resident on SSD (200 GB for 1 km² at 1 cm)

6 GB working set in VRAM (~200 chunks of 16³ voxels)

2-bit OccupancyBitmap (4 MB) co-resident: bit 0 = occupied,
bit 1 = render-flag

Per frame in the runtime renderer:

1. Clear render-flag bits (masked AND, ~10 μ s)
2. For each pixel:
 - Raymarch via DDA through voxel grid
 - On first occupied hit: imageAtomicOr to set render-flag, read voxel RGBA from atlas address (same address as flag, via UVW bijection)
 - Write pixel color
3. Read render-flag bitmap → derive touched-chunks set
4. Streamer: queue new chunks, evict cold chunks (with hysteresis)

Win for stock Lyra 2

Simpler graph. Stock Lyra 2 is one big neural network. No streaming, no atlas, no auxiliary bitmaps. Easier to reason about, easier to deploy in restricted environments where extra moving parts are a liability.

Win for Lyra 2 Lite

The architectural innovations are independent and stackable.

- UVW bijection: changes conditioning, doesn't require retraining
- OccupancyBitmap: enables demand streaming, doesn't touch the model
- CPU-first quantize: makes consumer hardware viable
- Octant pre-filter: free speedup, no quality cost
- Entity vocabulary: optional auxiliary conditioning
- TeaCache (when added): content-aware step skipping

Any subset can be cherry-picked. NVIDIA's PR review can accept the octant pre-filter (immediate merge candidate) while continuing to debate the canonical-encoding change.

The roadmap — what's coming

In PR #61 (filed against `nv-tlabs/lyra`, accepted commits TBD)

8 commits as of 2026-05-20:

1. RFC: World-coord canonical encoding via bidirectional UVW-RGB atlas
2. Add OccupancyBitmap class for early-skip of empty voxels
3. Add consumer-GPU --low-vram path (int8/int4/fp8)
4. Add streaming requantize — run conversion on consumer hardware
5. Pre-bake UVW bijection metadata during streaming requantize
6. Scale UVW metadata defaults to Lyra 2's 90m walkable spec
7. Extend OccupancyBitmap to 2-bit (occ+touched) and add CPU-first low-vram path
8. Add voxel-coord (UVW bijection) helpers to OccupancyBitmap

Just-added cherry-picks (PR #61 commits 9-12)

(Note: `octant_prefilter` was already in commit 1 — the RFC; these are four genuinely new additions.)

- **Commit 9:** AV1/HEVC default for inference scripts (better motion-vector fidelity for downstream tools)
- **Commit 10:** GLSL bindings module for OccupancyBitmap 2-bit pattern (ready-to-paste shader snippets)
- **Commit 11:** Optional entity-structured prompt hook (default off; +10% quality when used)
- **Commit 12:** Demand-streaming reference loop in OccupancyBitmap (per-frame helper showing the clear/render/read/update pattern)

Roadmap items NOT in PR #61 yet (planned)

- `torch.compile` + flash-attn 2.8.3+ integration (~1 week eng)
- TeaCache integration shim for Wan-2.1-derived DiT (~1 week eng)
- Full fp8 compute path via `transformer_engine` (~1 week eng)
- 2:4 structured sparsity (lossy, ~1-2 days eng)
- MotionAtlas sibling class for codec-MV-driven 3D scene flow (~1 day eng)

- DiT conditioning channel for motion histograms (research, ~1 month + cloud)
 - ← SCENE DEMO
 - UVW EXPLAINER
 - SOURCE ON GITHUB
- PyramidalWan-style multi-resolution refinement (research, ~1 month + cloud)

The bottom line

Question	Answer
Can I run Lyra 2 on my gaming PC?	Only with Lyra 2 Lite. Stock Lyra 2 needs a datacenter GPU.
Will I lose quality going Lite?	~5% on synthetic metrics, invisible in practice. Structured prompts actually <i>improve</i> quality.
Will my Lyra 2 outputs port to Lyra 2 Lite?	Yes — same model, same weights.
Will my Lyra 2 Lite outputs port to stock Lyra 2?	Yes — same model, same weights.
Can I commercially ship content baked with either?	Currently no — weights are NVIDIA research-only. Apache-2.0 alternative path documented.
Can I trust Lyra 2 Lite for production?	Yes, for the use cases it's designed for (offline content generation + consumer-GPU exploration). Edge cases still need stock Lyra 2 quality.
Should I file a PR to upstream NVIDIA?	PR #61 already there with 12 commits. Some pieces immediately mergeable (octant prefilter, GLSL bindings) without retraining.

Win for stock Lyra 2

It is the model. It exists because NVIDIA built it. The whole conversation only happens because they made it. Credit where due: this is research-quality, lab-tested, validated work that pushed the photo-to-3D state of the art forward by a meaningful margin.

Win for Lyra 2 Lite

It makes the model usable. Stock Lyra 2 was a tech demo inaccessible to 99% of developers. Lyra 2 Lite is what turns it into a tool that an individual hobbyist can run, iterate on, and build with. The

PR #61 contributions back upstream complete the loop — NVIDIA's research artifact becomes a developer tool that anyone can pick up.

← SCENE DEMO

UVW EXPLAINER

SOURCE ON GITHUB

And the honest meta-win

Both win when both exist. Lyra 2 stock is the canonical reference; Lyra 2 Lite is the consumer-accessible variant. They reinforce each other. NVIDIA's research validates the architecture; Lyra 2 Lite proves the architecture works on hardware the rest of the world owns.

The two-repo structure (`nv-tlabs/lyra` upstream + `MiL083/lyra` fork with PR #61) is the correct expression of this relationship. Code changes flow both ways. Architectural debates happen in the open. Neither owns the other.

That's the win condition for the whole effort.

Document last updated 2026-05-20. Author: MiLO + Opie. License: MIT. The companion document `DUMMIES.md` provides a metaphor-first introduction; this document is the technical comparison. The `LYRA2_PROPOSAL.md` is the canonical RFC for the architectural changes; this document summarizes the implications.